

PROJECT REPORT

VITyarthi - Build Your Own Project

DRONE FLIGHT CALCULATOR

A modular Python command-line tool for estimating drone flight time, range, battery usage and speed

Submitted by

Yashwanth Krishna

Register Number: 26BAS10066

Language: Python 3 | Interface: Console | Academic Year: 2026

Table of Contents

1. Cover Page
 2. Introduction
 3. Problem Statement
 4. Functional Requirements
 5. Non-functional Requirements
 6. System Architecture
 7. Design Diagrams
 8. Design Decisions and Rationale
 9. Implementation Details
 10. Screenshots / Results
 11. Testing Approach
 12. Challenges Faced
 13. Learnings and Key Takeaways
 14. Future Enhancements
 15. References
- Appendix A: Source Code

2. Introduction

Unmanned Aerial Vehicles (UAVs), commonly called drones, are used for photography, surveying, agriculture, delivery and education. Whether a mission is possible depends mainly on the battery: how much energy is stored, how quickly the motors consume it, and how far the drone can travel in the time available. These questions are answered with a handful of standard formulas involving energy (Wh), power (W), speed (km/h), distance (km) and time.

Drone Flight Calculator is a menu-driven Python application that packages these formulas into four easy-to-use calculators. The user only has to type the values printed on the battery and motor datasheets (capacity in mAh, voltage in V, power in W, speed in km/h, and so on); the program performs all unit conversions, validates the inputs and prints clear, rounded results.

The project applies core programming concepts: functions and modules, user input and output, arithmetic and unit conversion, conditional statements, loops, and input validation.

2.1 Objectives

- Estimate drone flight time from battery capacity, voltage and power consumption.
- Estimate flight range from speed and flight time.
- Calculate battery energy, energy used and the percentage of battery consumed and remaining for a planned flight.
- Provide a general speed / distance / time calculator.
- Build the tool as separate, reusable modules with validation and a simple menu-based workflow.

2.2 Scope and Target Users

The tool gives first-order estimates assuming constant power draw and constant speed. It is intended for drone hobbyists, students and teams planning flights, and for educational demonstration of energy and kinematics calculations. It is not a substitute for flight-controller telemetry or manufacturer specifications.

3. Problem Statement

Drone operators frequently need to know how long a battery will last, how far the drone can fly, and how much charge a planned flight will use. Doing this by hand is error-prone because the inputs come in mixed units (mAh, minutes, km/h) that must be converted before the formulas can be applied, and a small slip can lead to an over-optimistic estimate and a drone that runs out of charge in the air.

There is a need for a simple, dependency-free tool that accepts the values operators already know, converts units automatically, rejects invalid values, and reports flight time, range, battery usage and speed/distance/time in one place.

4. Functional Requirements

The system consists of four major functional modules (the guideline requires at least three), a clear input/output structure and a logical user workflow.

ID	Module	Inputs	Processing	Output
FR1	Flight Time Calculator (flight_time.py)	Capacity (mAh), voltage (V), power (W)	Ah = mAh/1000; Energy (Wh) = V x Ah; Time (h) = Energy / Power; minutes = h x 60	Battery energy (Wh); estimated flight time (min)
FR2	Range Calculator (range_calculator.py)	Speed (km/h), flight time (min)	Hours = min/60; Distance = Speed x Hours	Estimated range (km)
FR3	Battery Calculator (battery_calculator.py)	Capacity (mAh), voltage (V), flight time (min), power (W)	Energy = V x Ah; Used = P x (min/60); Used % = Used/Energy x 100 (max 100); Remaining % = 100 - Used %	Battery energy, energy used, % used, % remaining
FR4	Speed Calculator (speed_calculator.py)	Sub-menu: (1) distance, time; (2) speed, time; (3) distance, speed	Speed = D/T; Distance = S x T; Time = D/S	Speed (km/h), distance (km) or time (h)
FR5	Main Menu (main.py)	Menu choice 1-5	Loop; dispatch to module; exit on 5; reject other choices	Menu, module output, messages
FR6	Input validation (all modules)	All numeric values	Reject values <= 0	'Values must be greater than zero.'

4.1 Input / Output Structure

All input is read from the keyboard through `input()` and converted with `float()`. All output is printed on the console and rounded to two decimal places. Each calculator shows a banner, prompts for each value with its unit, and prints labelled results, for example Estimated Flight Time: 22.2 minutes.

5. Non-functional Requirements

The following non-functional requirements were defined for the project, together with their current status in the implementation.

Requirement	Description	Status in current code
Usability	Simple numbered menus, unit shown in every prompt, labelled results with units.	Met
Performance	Each calculation is a constant-time arithmetic operation; results appear instantly.	Met
Reliability / Validation	Zero and negative values must be rejected; overuse of battery must not give negative remaining percentage.	Met (values <= 0 rejected; used % capped at 100)
Maintainability	One file per calculator, one function per file, separate entry point.	Met
Portability	Runs on Windows, Linux and macOS with Python 3 and no third-party libraries.	Met
Resource efficiency	No files, network or database; negligible memory and CPU use.	Met
Error-handling strategy	Non-numeric input should show a friendly message instead of a traceback.	Partial - planned (see Sections 12 and 14)

Requirement	Description	Status in current code
Logging / monitoring	Optional log of calculations and errors.	Not implemented - planned

6. System Architecture

The application follows a simple two-layer architecture. The **presentation layer** (`main.py`) displays the menu, reads the user's choice and calls the relevant function. The **business-logic layer** consists of four independent calculator modules; each one collects its inputs, validates them, applies the formulas and prints the result. Control always returns to the main menu, so several calculations can be done in one session. The project uses no database or file storage, so all data exists only during a calculation.

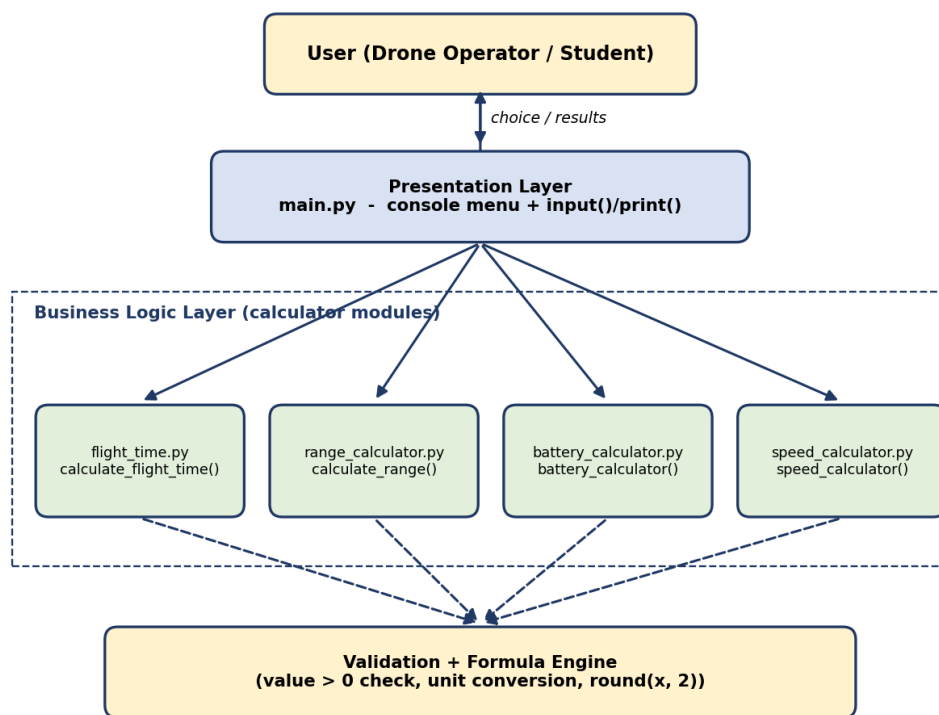


Figure 1: System architecture of the Drone Flight Calculator

7. Design Diagrams

7.1 Use Case Diagram

The only actor is the User (drone operator or student). Each calculator is a use case, and all of them include entering and validating input values.

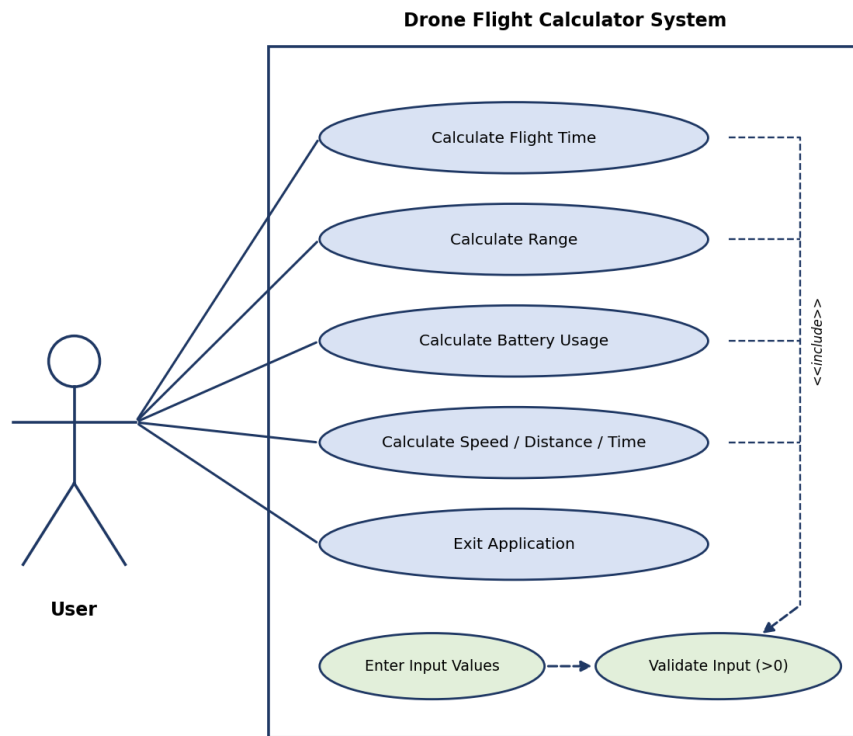


Figure 2: Use case diagram

7.2 Workflow Diagram

The workflow shows the main loop: the menu is displayed, the choice is read and either dispatched to a calculator, rejected as invalid, or used to exit. Inside a calculator, invalid values end the function with an error message; valid values lead to calculation and output.

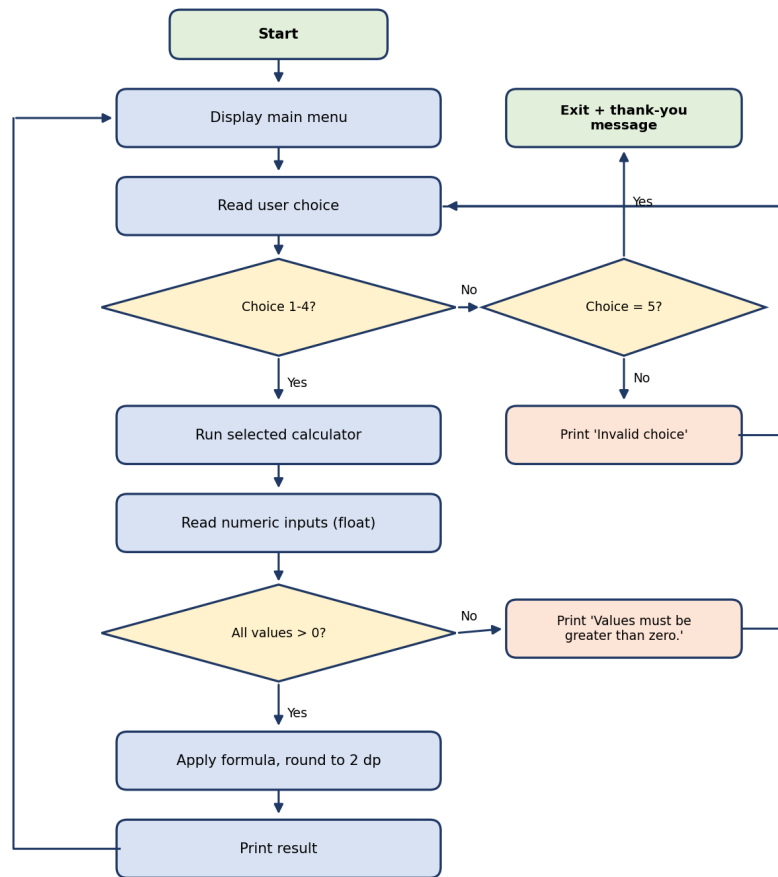


Figure 3: Workflow (flowchart) of the application

7.3 Sequence Diagram

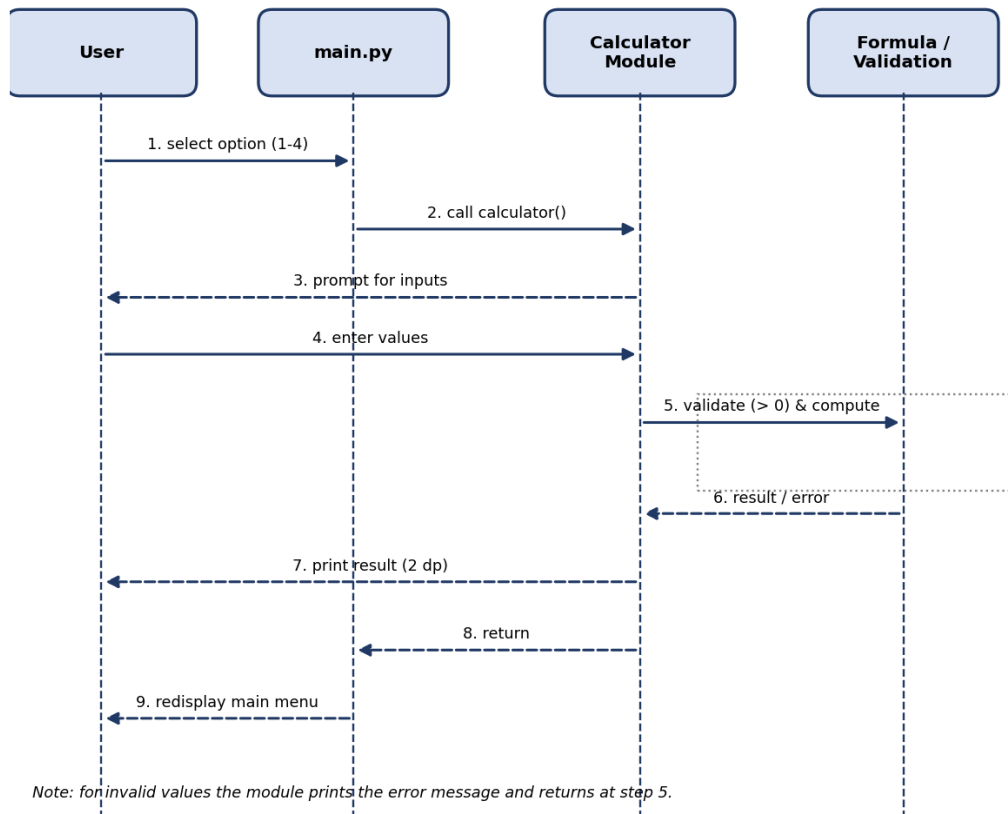


Figure 4: Sequence diagram for one calculation

7.4 Class / Component Diagram

The project is procedural, so it is shown as a component diagram: `main.py` imports and calls one function from each calculator module.

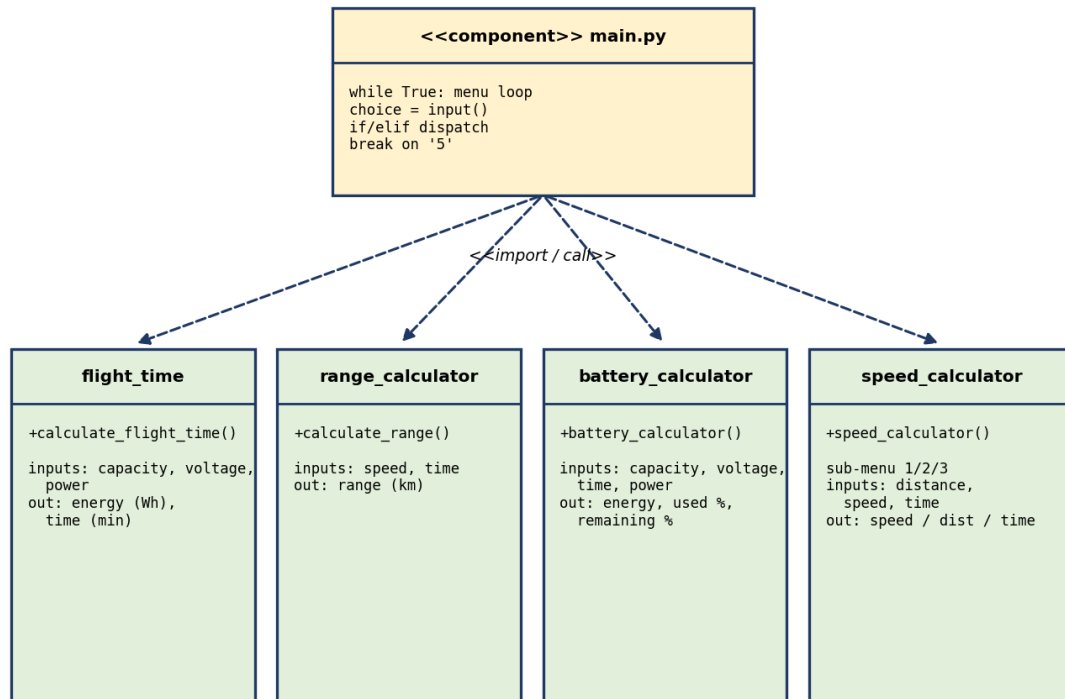


Figure 5: Component diagram

7.5 ER Diagram

Not applicable. The application does not store data in a database or file, so no ER diagram or schema is required. This is noted as a possible enhancement (calculation history) in Section 14.

8. Design Decisions and Rationale

Decision	Rationale
Python with only the standard library	Readable, portable and easy to run anywhere; no installation of packages.
One module (file) per calculator	Separation of concerns; each calculator can be tested, changed or reused without touching others; new calculators can be added easily.
Separate entry point (main.py) with a while-loop menu	Lets the user perform many calculations in one session and exit deliberately.
Ask for capacity in mAh and time in minutes	These are the units printed on batteries and used by pilots; the program handles conversion to Ah and hours, which removes a common source of mistakes.
Energy-based model ($Wh = V \times Ah$; $time = Wh / W$)	Standard, simple and accurate for first-order estimates of battery endurance.
Validation of values > 0 before calculating	Prevents division by zero and meaningless negative results.
Cap 'battery used' at 100% and remaining at 0%	A flight that needs more energy than the battery holds should show an empty battery, not a negative percentage.
Round output to two decimals	Keeps results readable without hiding meaningful precision.

9. Implementation Details

9.1 Project Structure

```
drone-flight-calculator/  
|-- main.py           # menu loop / entry point  
|-- flight_time.py    # calculate_flight_time()  
|-- range_calculator.py # calculate_range()  
|-- battery_calculator.py # battery_calculator()  
|-- speed_calculator.py # speed_calculator()
```

9.2 main.py

Imports the four functions and runs an infinite `while True` loop. An `if / elif` chain maps choices '1' to '4' to the calculators, '5' prints a thank-you message and executes `break`, and any other input prints 'Invalid choice'.

9.3 flight_time.py

Reads capacity, voltage and power, validates them, and computes:

```
capacity_ah = capacity / 1000  
battery_energy = voltage * capacity_ah          # Wh  
flight_time_hours = battery_energy / power  
flight_time_minutes = flight_time_hours * 60
```

9.4 range_calculator.py

```
flight_time_hours = flight_time / 60  
distance = speed * flight_time_hours           # km
```

9.5 battery_calculator.py

Computes total battery energy, energy used during the planned flight and the percentages. The used percentage is limited to 100.

```
battery_energy = capacity_ah * voltage  
energy_used = power * (flight_time / 60)  
battery_used_percent = (energy_used / battery_energy) * 100  
if battery_used_percent > 100:  
    battery_used_percent = 100  
remaining_percent = 100 - battery_used_percent
```

9.6 speed_calculator.py

Shows a sub-menu (1 speed, 2 distance, 3 time). Each branch reads two values, validates them and applies one of `speed = distance / time`, `distance = speed * time`, `time = distance / speed`.

9.7 Validation and Output

Every function contains a check of the form `if a <= 0 or b <= 0: print('Values must be greater than zero.');` `return`, which stops the calculation early and returns to the menu. Results are formatted with `round(value, 2)` and printed with their units. The complete source code is in Appendix A.

10. Screenshots / Results

The following console sessions were produced by running the project code with the sample inputs shown (a 5000 mAh, 11.1 V battery, 150 W power draw).

```
=====
      DRONE FLIGHT CALCULATOR
=====
1. Flight Time Calculator
2. Range Calculator
3. Battery Calculator
4. Speed Calculator
5. Exit
=====
Enter your choice: 1

----- FLIGHT TIME CALCULATOR -----
Enter battery capacity (mAh): 5000
Enter battery voltage (V): 11.1
Enter power consumption (W): 150

Battery Energy: 55.5 Wh
Estimated Flight Time: 22.2 minutes
```

Figure 6: Flight Time Calculator - 55.5 Wh battery gives 22.2 minutes

```
=====
      DRONE FLIGHT CALCULATOR
=====
1. Flight Time Calculator
2. Range Calculator
3. Battery Calculator
4. Speed Calculator
5. Exit
=====
Enter your choice: 2

----- RANGE CALCULATOR -----
Enter drone speed (km/h): 60
Enter flight time (minutes): 22.2

Estimated Flight Range: 22.2 km
```

Figure 7: Range Calculator - 60 km/h for 22.2 minutes gives 22.2 km

```

=====
DRONE FLIGHT CALCULATOR
=====
1. Flight Time Calculator
2. Range Calculator
3. Battery Calculator
4. Speed Calculator
5. Exit
=====
Enter your choice: 3

----- BATTERY CALCULATOR -----
Enter battery capacity (mAh): 5000
Enter battery voltage (V): 11.1
Enter flight time (minutes): 15
Enter power consumption (W): 150

Battery Energy: 55.5 Wh
Energy Used: 37.5 Wh
Battery Used: 67.57 %
Remaining Battery: 32.43 %

```

Figure 8: Battery Calculator - a 15-minute flight uses 67.57% of the battery

```

=====
DRONE FLIGHT CALCULATOR
=====
1. Flight Time Calculator
2. Range Calculator
3. Battery Calculator
4. Speed Calculator
5. Exit
=====
Enter your choice: 4

----- SPEED CALCULATOR -----
1. Calculate Speed
2. Calculate Distance
3. Calculate Time
Enter your choice: 2
Enter speed (km/h): 40
Enter time (hours): 0.75

Distance: 30.0 km

```

Figure 9: Speed Calculator (option 2) - distance = 40 km/h x 0.75 h = 30 km

```

=====
          DRONE FLIGHT CALCULATOR
=====
1. Flight Time Calculator
2. Range Calculator
3. Battery Calculator
4. Speed Calculator
5. Exit
=====
Enter your choice: 2

----- RANGE CALCULATOR -----
Enter drone speed (km/h): 0
Enter flight time (minutes): 30
Values must be greater than zero.

=====
          DRONE FLIGHT CALCULATOR
=====
1. Flight Time Calculator
2. Range Calculator
3. Battery Calculator
4. Speed Calculator
5. Exit
=====
Enter your choice: 9

Invalid choice. Please try again.

```

Figure 10: Validation - zero speed rejected, and invalid menu choice handled

11. Testing Approach

Testing was done as black-box validation testing: the program was run with prepared inputs through its menu, and the actual output was compared with the expected value calculated by hand from the formulas. Normal, boundary and invalid inputs were all tried.

#	Module	Input	Expected	Actual	Result
1	Flight time	5000 mAh, 11.1 V, 150 W	55.5 Wh; 22.2 min	55.5 Wh; 22.2 min	Pass
2	Range	60 km/h, 22.2 min	22.2 km	22.2 km	Pass
3	Battery	5000 mAh, 11.1 V, 15 min, 150 W	55.5 Wh; 37.5 Wh; 67.57%; 32.43%	Same	Pass
4	Battery (overuse)	2000 mAh, 11.1 V, 30 min, 200 W	Used capped at 100%; remaining 0%	100 %; 0 % (Energy Used shows 100.0 Wh, more than 22.2 Wh battery)	Pass (see Sec. 12)
5	Speed opt. 1	10 km, 0.5 h	20 km/h	20.0 km/h	Pass
6	Speed opt. 2	40 km/h, 0.75 h	30 km	30.0 km	Pass
7	Speed opt. 3	30 km, 60 km/h	0.5 h	0.5 hours	Pass
8	Flight time	capacity = -5000	Error message	'Values must be greater than zero.'	Pass
9	Range	speed = 0	Error message	'Values must be greater than zero.'	Pass

#	Module	Input	Expected	Actual	Result
10	Main menu	choice = 9	Invalid choice message	'Invalid choice. Please try again.'	Pass
11	Main menu	choice = 5	Thank-you and exit	Message printed, program ends	Pass
12	Flight time	capacity = 'abc'	Friendly error	Program stops with ValueError traceback	Fail - known limitation

Eleven of twelve tests behaved as required. The failing case identifies the main improvement needed (exception handling for non-numeric input). As future work, these cases can be converted into automated unit tests with Python's `unittest` or `pytest`.

12. Challenges Faced

- **Unit conversion:** battery capacity is given in mAh and time in minutes, while the formulas need Ah and hours. Converting inside each module ($\text{capacity} / 1000$, $\text{minutes} / 60$) solved this.
- **Battery overuse:** when the planned flight needs more energy than the battery holds, the percentage would exceed 100 and the remaining battery become negative. The used percentage is therefore capped at 100 and the remaining shown as 0. Testing showed that 'Energy Used' is still printed uncapped (100 Wh from a 22.2 Wh battery); this is intentional, since it shows how much energy the flight actually needs, but it could be labelled more clearly.
- **Invalid values:** zero or negative values cause division by zero or meaningless results, so every module validates its inputs before calculating.
- **Non-numeric input:** converting text with `float()` raises a `ValueError` that is not yet caught. This is the main open limitation and is planned as the first enhancement.
- **Modular design:** organising the project into separate files while keeping one common menu required careful naming of functions and imports.
- **Assumptions:** the model assumes constant power and speed and a fully usable battery, whereas real flights vary with wind, payload and battery health.

13. Learnings and Key Takeaways

- How to break a problem into small modules and connect them with a central menu.
- How to translate physical formulas (energy, power, speed) into code, including careful unit handling.
- The importance of input validation and of testing boundary and invalid cases, not only normal inputs.
- That user-facing programs must handle unexpected input gracefully, and that testing reveals gaps such as unhandled exceptions.
- How clear prompts, units and labelled results make a console program easier to use.
- The value of documentation, design diagrams and version control when presenting and maintaining a project.

14. Future Enhancements

- Add try-except handling and re-prompting loops for non-numeric input.
- Add automated unit tests and a tests/ folder; use Git for version control with regular commits.
- Add logging of calculations and errors to a file.
- Include a safety reserve (for example 20%) in flight time and range, and a round-trip range option.
- Estimate power consumption from drone weight, payload and motor data.
- Store calculation history (CSV or SQLite), which would also add an ER diagram and schema.
- Provide a graphical or web interface, unit selection (m/s, miles, knots) and battery cell-count presets.
- Merge shared validation code into a common utilities module.

15. References

- Python Software Foundation, *Python 3 Documentation*, <https://docs.python.org/3/>
- Halliday, D., Resnick, R. and Walker, J., *Fundamentals of Physics*, Wiley (energy, power, speed, distance and time relationships).
- LiPo battery datasheets (capacity in mAh, nominal voltage per cell) from battery manufacturers.
- VITyarthi, *Build Your Own Project - General Project Instructions and Submission Guidelines*.

Appendix A: Source Code

main.py

```
from flight_time import calculate_flight_time
from range_calculator import calculate_range
from battery_calculator import battery_calculator
from speed_calculator import speed_calculator

while True:
    print("\n=====")
    print("          DRONE FLIGHT CALCULATOR")
    print("=====")
    print("1. Flight Time Calculator")
    print("2. Range Calculator")
    print("3. Battery Calculator")
    print("4. Speed Calculator")
    print("5. Exit")
    print("=====")

    choice = input("Enter your choice: ")

    if choice == "1":
        calculate_flight_time()

    elif choice == "2":
        calculate_range()

    elif choice == "3":
        battery_calculator()

    elif choice == "4":
        speed_calculator()

    elif choice == "5":
        print("\nThank you for using Drone Flight Calculator!")
        break

    else:
        print("\nInvalid choice. Please try again.")
```

flight_time.py

```
def calculate_flight_time():
    print("\n----- FLIGHT TIME CALCULATOR -----")

    capacity = float(input("Enter battery capacity (mAh): "))
    voltage = float(input("Enter battery voltage (V): "))
    power = float(input("Enter power consumption (W): "))

    if capacity <= 0 or voltage <= 0 or power <= 0:
        print("Values must be greater than zero.")
        return

    capacity_ah = capacity / 1000

    battery_energy = voltage * capacity_ah

    flight_time_hours = battery_energy / power
    flight_time_minutes = flight_time_hours * 60

    print("\nBattery Energy:", round(battery_energy, 2), "Wh")
    print("Estimated Flight Time:",
          round(flight_time_minutes, 2), "minutes")
```

range_calculator.py

```
def calculate_range():
    print("\n----- RANGE CALCULATOR -----")

    speed = float(input("Enter drone speed (km/h): "))
```

```

flight_time = float(input("Enter flight time (minutes): "))

if speed <= 0 or flight_time <= 0:
    print("Values must be greater than zero.")
    return

flight_time_hours = flight_time / 60

distance = speed * flight_time_hours

print("\nEstimated Flight Range:",
      round(distance, 2), "km")

```

battery_calculator.py

```

def battery_calculator():
    print("\n----- BATTERY CALCULATOR -----")

    capacity = float(input("Enter battery capacity (mAh): "))
    voltage = float(input("Enter battery voltage (V): "))
    flight_time = float(input("Enter flight time (minutes): "))
    power = float(input("Enter power consumption (W): "))

    if capacity <= 0 or voltage <= 0 or flight_time <= 0 or power <= 0:
        print("Values must be greater than zero.")
        return

    capacity_ah = capacity / 1000

    battery_energy = capacity_ah * voltage

    energy_used = power * (flight_time / 60)

    remaining_energy = battery_energy - energy_used

    if remaining_energy < 0:
        remaining_energy = 0

    battery_used_percent = (energy_used / battery_energy) * 100

    if battery_used_percent > 100:
        battery_used_percent = 100

    remaining_percent = 100 - battery_used_percent

    print("\nBattery Energy:",
          round(battery_energy, 2), "Wh")

    print("Energy Used:",
          round(energy_used, 2), "Wh")

    print("Battery Used:",
          round(battery_used_percent, 2), "%")

    print("Remaining Battery:",
          round(remaining_percent, 2), "%")

```

speed_calculator.py

```

def speed_calculator():
    print("\n----- SPEED CALCULATOR -----")
    print("1. Calculate Speed")
    print("2. Calculate Distance")
    print("3. Calculate Time")

    choice = input("Enter your choice: ")

    if choice == "1":
        distance = float(input("Enter distance (km): "))
        time = float(input("Enter time (hours): "))

        if distance <= 0 or time <= 0:
            print("Values must be greater than zero.")
            return

```

```

    speed = distance / time

    print("\nSpeed:",
          round(speed, 2), "km/h")

elif choice == "2":
    speed = float(input("Enter speed (km/h): "))
    time = float(input("Enter time (hours): "))

    if speed <= 0 or time <= 0:
        print("Values must be greater than zero.")
        return

    distance = speed * time

    print("\nDistance:",
          round(distance, 2), "km")

elif choice == "3":
    distance = float(input("Enter distance (km): "))
    speed = float(input("Enter speed (km/h): "))

    if distance <= 0 or speed <= 0:
        print("Values must be greater than zero.")
        return

    time = distance / speed

    print("\nTime:",
          round(time, 2), "hours")

else:
    print("Invalid choice.")

```